

Roadmapa libGDX – krok dalej (po ukończeniu oficjalnych tutoriali)

Zakładam, że masz już za sobą "A Simple Game" i "Simple Game Extended" – czyli znasz `Game`, `Screen`, `SpriteBatch`, `Texture`, podstawowy input i pętlę `render()`. Ta roadmapa NIE powtarza tamtego materiału – wchodzi od razu w rzeczy, których tam nie było albo były tylko dotknięte powierzchownie.

Szacowany czas: 3-4 tygodnie przy 1-2h dziennie.

Etap 1: Kamera i viewport "na twardo" – zrozum problem, zanim użyjesz rozwiązania (3-4 dni)

To jest dokładnie to, o czym myślałeś – i słusznie, to najważniejsza luka po tutorialach.

Krok A – zrób to źle, żeby zobaczyć problem:

1. Weź swój projekt z tutoriala (albo nowy, prosty ekran z jedną teksturą na środku).
2. NIE używaj żadnego `Viewport`. Stwórz samą `OrthographicCamera` z ustawionym `viewportWidth/Height` = rozmiar okna przy starcie.
3. W `resize(width, height)` zrób to, co najbardziej intuicyjne dla kogoś bez wiedzy o viewportach: ustaw `camera.viewportWidth = width; camera.viewportHeight = height; camera.update();`
4. Zmień rozmiar okna (przeciągnij za róg) i zobacz co się dzieje z Twoją grafiką – prawdopodobnie się rozciągnie albo przesunie nienaturalnie, bo mieszasz piksele okna z jednostkami świata gry.
5. Zanotuj (samemu sobie), dlaczego to jest złe – to ważniejsze niż sama naprawa.

Krok B – rozwiąż to poprawnie:

1. Zdefiniuj stały rozmiar świata gry, niezależny od rozmiaru okna, np. `WORLD_WIDTH = 800, WORLD_HEIGHT = 480`.
2. Podłącz `FitViewport(WORLD_WIDTH, WORLD_HEIGHT, camera)`.
3. W `resize()` wywołaj tylko `viewport.update(width, height)` – i porównaj z tym, co robiłeś ręcznie w kroku A.
4. Przetestuj zmianę rozmiaru okna – proporcje powinny zostać zachowane (czarne pasy po bokach/górze/dole zamiast rozciągania).
5. Podmień `FitViewport` na `StretchViewport` i `ExtendViewport` – zobacz na oczy różnicę (rozciąganie vs. dodatkowa widoczna przestrzeń).

Krok C – kamera podążająca za obiektem:

1. Zbuduj świat gry większy niż ekran (np. duże tło albo siatka referencyjna z liniami co 100px, żeby widzieć ruch).
2. Sterowalny kwadrat/sprite (masz to z tutoriala) – niech kamera podąża za nim:
`camera.position.set(player.x, player.y, 0)`.
3. Dodaj ograniczenie kamery, żeby nie pokazywała obszaru poza granicami mapy (clamp pozycji kamery do rozmiaru świata minus połowa viewportu).
4. Zamień na płynne podążanie (lerp) zamiast sztywnego "przyklejenia":
`camera.position.lerp(target, 0.1f)`.

Kryterium ukończenia: potrafisz wyjaśnić komuś innemu, dlaczego samo ustawienie `viewportWidth/Height` w `resize()` nie wystarcza, i masz działającą kamerę śledzącą gracza w większym świecie z ograniczeniami.

Etap 2: Współrzędne świata vs. ekranu vs. UI (2 dni)

To naturalne rozszerzenie etapu 1 – często mylone przez początkujących.

Teoria:

- Różnica między współrzędnymi świata gry (world units, w których poruszają się obiekty) a współrzędnymi ekranu (piksele okna).
- `viewport.unproject()` / `viewport.project()` – zamiana współrzędnych myszy/dotyku (piksele ekranu) na współrzędne świata gry.
- Dlaczego UI (HUD, punkty, przyciski) zwykle chce **osobnej** kamery/viewportu niż świat gry (żeby UI nie skalowało się razem z zoomem kamery gry).

Zadanie praktyczne:

1. Kliknij na ekranie – zamień kliknięcie (piksele) na pozycję w świecie gry przez `viewport.unproject(new Vector3(screenX, screenY, 0))`.
2. Narysuj mały znacznik dokładnie tam, gdzie kliknąłeś, w świecie gry (nie na ekranie).
3. Dodaj drugą kamerę + `FitViewport` tylko do UI (np. licznik punktów w rogu ekranu) i upewnij się, że nie rusza się razem z kamerą świata, gdy ta podąża za graczem.

Kryterium ukończenia: rozumiesz i potrafisz zademonstrować różnicę między kliknięciem na ekranie a pozycją w świecie gry, oraz masz osobną kamerę do UI.

Etap 3: Kolizje i prosta struktura gry (3-4 dni)

Jeśli tutoriale dotknęły tego tylko pobieżnie – czas to ugruntować.

Teoria:

- `Rectangle.overlaps()` – AABB.
- Lista obiektów gry (`com.badlogic.gdx.utils.Array`), nie `ArrayList` – warto się przyzwyczajać do kolekcji libGDX) + pętla update/render.
- Bezpieczne usuwanie obiektów podczas iteracji (np. iterator, albo kolejka do usunięcia po pętli).

Zadanie praktyczne - "Zbieracz monet" w Twoim większym świecie z etapu 1:

1. Wygeneruj 10-15 monet rozrzuconych po świecie gry (nie po ekranie!).
2. Gracz zbiera monetę przy kolizji – moneta znika, licznik punktów rośnie.
3. Wyświetl licznik na osobnej kamerze UI z etapu 2 (`BitmapFont` albo `Scene2D Label`), jeśli już chcesz przeskoczyć).
4. Dodaj statyczne przeszkody (ściany) blokujące ruch gracza.

Kryterium ukończenia: masz pętlę ruch → kolizja → reakcja → aktualizacja UI, działającą w świecie większym niż ekran, z poprawnie skalującym się widokiem.

Etap 4: Animacje i dźwięk (2-3 dni)

Teoria:

- `TextureRegion` i `Animation<TextureRegion>`.
- `TextureAtlas` + `TexturePacker`.
- `Sound` vs `Music`.

Zadanie praktyczne:

1. Dodaj spritesheet postaci (kenney.nl / opengameart.org) z animacją chodzenia w 4 kierunkach, zależną od kierunku ruchu gracza.
2. Dźwięk przy zbieraniu monety.
3. Muzyka w tle w pętli, z wyciszeniem pod klawiszem.

Kryterium ukończenia: postać animuje się poprawnie zależnie od kierunku ruchu, gra ma dźwięk i muzykę.

Etap 5: AssetManager i wieloekranowość (3-4 dni)

Teoria:

- `AssetManager` – ładowanie asynchroniczne zamiast `new Texture(...)` wszędzie.
- `Game` + wiele `Screen` – menu, gra, game over (masz to częściowo z tutoriala – teraz dodaj ekran ładowania).

Zadanie praktyczne:

1. Przenieś ładowanie tekstur/dźwięków na `AssetManager`.
2. Dodaj `LoadingScreen` z paskiem postępu (`assetManager.getProgress()`).
3. `MainMenuScreen` → `GameScreen` → `GameOverScreen`, każdy jako osobna klasa `Screen`.

Kryterium ukończenia: zasoby ładują się przez `AssetManager` z ekranem ładowania, przejścia między ekranami działają płynnie.

Projekt spinający (tydzień 4): połącz wszystko

Rozbuduj mechanikę z Etapu 3 do małej, kompletnej gry:

- Kamera śledząca gracza w większym świecie (Etap 1) + osobna kamera UI (Etap 2).
- Kolizje z monetami i przeszkodami (Etap 3).
- Animowana postać + dźwięk/muzyka (Etap 4).
- Menu, ekran gry, ekran końca gry, `AssetManager` (Etap 5).
- Zapis najlepszego wyniku (`Preferences`).

To zamyka "podstawy" – od tego miejsca możesz iść w **Box2D** (prawdziwa fizyka), **Tiled** (mapy kafelkowe) albo **Scene2D UI** głębiej (jeśli chcesz ładniejsze menu niż gołe `BitmapFont`).

Co dalej (poza zakresem tej roadmapy)

- Box2D – `World`, `Body`, `Fixture`, `ContactListener`.
- Tiled + `TmxMapLoader` – mapy kafelkowe, warstwy kolizji.
- Scene2D UI głębiej – `Stage`, `Table`, `Skin` do rozbudowanego HUD/menu.
- ECS (Ashley) – gdy projekt zacznie robić się zbyt "spaghetti" z klasami obiektów.

Przydatne źródła

- libgdx.com/wiki – sekcje "Viewports" i "Orthographic Camera" (przeczytaj po Etapie 1, żeby zweryfikować swoje zrozumienie).
 - [Kenney.nl](http://kenney.nl), opengameart.org – darmowe assety.
 - "GamesFromScratch" i "Brent Aureli" na YouTube.
-

Wskazówka dot. tempa

Etap 1 (kamera/viewport) jest tu najważniejszy i celowo rozbity na "zrób źle → zobacz

problem → napraw" - to lepiej buduje intuicję niż czytanie o `FitViewport` i przyjęcie go na wiarę. Nie spiesz się z tym etapem, reszta roadmapy jest już prostsza.